

Código y artefactos reproducibles:

<https://github.com/Couyoumdjian13/Analisis-Last.fm-1K-Users>

Resumen — En este hito consolidamos un pipeline reproducible para Last.fm 1K Users (preprocesamiento en ≈ 67 s y memoria por debajo de 1 GB) y migramos la evaluación a un protocolo temporal leave-one-out coherente con el paradigma repeat-aware. Sobre el subset H1 (200,000 interacciones, 100 usuarios y 88,234 ítems) evaluamos cuatro modelos de referencia y formalizamos matemáticamente nuestras dos contribuciones propias: T-BPR (muestreo negativo dependiente del tiempo) y un Modelo Híbrido de ruteo Repeat/Explore. El hallazgo principal es que la frecuencia histórica por sí sola es una señal débil (Spearman $\rho = 0.284$): el único baseline con aciertos fue SimpleRepeat-Recency (Recall@10 = 0.0900), lo que confirma la hipótesis de localidad temporal y motiva el diseño de los modelos propios, foco central de H3.

1. Progreso en el Desarrollo de la Solución

1.1 Infraestructura de datos y reproducibilidad

Se reescribió el pipeline de preprocesamiento del dataset Last.fm 1K Users para permitir su ejecución autónoma y eficiente en memoria. El archivo TSV original de 2.5 GB (~ 19 millones de filas) saturaba la RAM en entornos con menos de 16 GB. La nueva implementación (notebooks/preprocessing.py) opera en dos pasadas secuenciales:

- Lectura por fragmentos (*chunked*) con pandas, donde se descartan los campos `artist_id` y `track_id` debido a la alta presencia de valores nulos (NaNs) de MusicBrainz, realizando una escritura incremental en un archivo Parquet comprimido con pyarrow. Para subsanar la falta de identificadores en más de 69,000 artistas, se construyó una clave única de ítem estructurada como `item_id = "<artist_name> - <track_name>"`. Esto evita la propagación de nulos en los *embeddings* posteriores.
- Lectura selectiva (*predicate pushdown*) del Parquet completo para extraer el subconjunto H1 con los 100 usuarios más activos.

Con este diseño, el tiempo de ejecución sobre los 19 millones de eventos se redujo a aproximadamente 67 segundos y el pico de memoria se estabilizó por debajo de 1 GB.

1.2 Temporal BPR: muestreo negativo dependiente del tiempo

El método de referencia para nuestra adaptación es Bayesian Personalized Ranking (BPR). Este método fue abordado en las clases del curso y como grupo entendimos mejor el gran potencial que tiene gracias a nuestra exposición del seminario. Si bien el problema del seminario era totalmente opuesto al abordado en este proyecto (Cold Start vs Repeated Aware) se encuentran nexos que respaldan su aplicación.

La función objetivo en cuestión es:

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2$$

En este BPR estándar, j se muestrea uniformemente desde el complemento del historial H_u . Sin embargo, nuestra adaptación, denominada T-BPR, reemplaza ese muestreo uniforme por una distribución dependiente del tiempo. Sea t el instante de la interacción positiva (u, i, t) y $\Delta_{uj}(t) = t - t_{uj}^{last}$ el intervalo desde la última vez que u consumió j (definido como $+\infty$ si $j \notin H_u$).

Luego se define una *ventana óptima de repetición* específica por usuario $W_u = (a_u, b_u)$, calibrada a partir de los percentiles 25 y 75 de la distribución empírica de intervalos entre repeticiones de u . Entonces la probabilidad de muestrear j como negativo se modula entonces como:

$$p(j \mid u, t) \propto \begin{cases} \alpha & \text{si } \Delta_{uj}(t) \in W_u \\ 1 & \text{si } j \notin H_u \\ \beta & \text{en otro caso,} \end{cases} \quad 0 < \alpha < 1 < \beta$$

De esta forma, si un ítem ya consumido cae dentro de la ventana en que el usuario suele repetir, no debe penalizarse como si fuera negativo (de ahí $\alpha < 1$); en cambio, ítems repetidos fuera de su ventana habitual se penalizan con peso $\beta > 1$. De esta forma se empuja al modelo a aprender el patrón temporal de repetición.

1.3 Modelo Híbrido: ruteo Repeat / Explore

A su vez, se mantiene una segunda opción de solución (en caso de encontrar futuros problemas con la primera opción). Esta consiste en una arquitectura de enrutamiento basada en un clasificador de segundo nivel. Este módulo evalúa dinámicamente cada par usuario-tiempo $(u, t + 1)$, para decidir si la recomendación debe delegarse a un sub-recomendador especializado en repetición (SimpleRepeat) o a uno enfocado en la exploración de nuevos ítems (filtrado colaborativo iALS). El clasificador se diseñó deliberadamente liviano (un árbol de decisión de profundidad acotada, del que se obtiene una probabilidad a partir de las proporciones de clase en sus hojas) para preservar su interpretabilidad y permitir el análisis de variables

Para cada interacción potencial se construye un vector de *features* $f_{u,t} \in \mathbb{R}^4$:

- $f_1 = r_u = \frac{|\{i \in H_u : c_{ui} > 1\}|}{|H_u|}$: tasa de repetición histórica del usuario.
- $f_2 = \log(1 + \tau_{u,t})$ con $\tau_{u,t}$ el tiempo (en horas) desde la última interacción registrada de u .
- $f_3 = \log(1 + \bar{\Delta}_u)$: log-intervalo medio entre repeticiones del usuario.
- $f_4 = |H_u^{30d}|$: número de interacciones en la ventana móvil de 30 días.

El clasificador estima $P(\text{modo} = \text{repeat} \mid f_{u,t})$; esta probabilidad actúa como peso de gating sobre ambos sub-rankings: SimpleRepeat-Recency recibe peso $w_{\text{repeat}} = P(\text{modo} = \text{repeat} \mid f_{u,t})$ y el modelo colaborativo (iALS) peso $w_{\text{collab}} = 1 - w_{\text{repeat}}$. La lista top-K final se obtiene como la unión de ambos sub-rankings ponderada por estos pesos; el umbral θ (ajustado por validación cruzada para optimizar nDCG@10) fija su punto de operación. Este diseño está inspirado en arquitecturas de *gating* utilizadas en sistemas de recomendación sensibles al contexto. La arquitectura propuesta se resume en el diagrama del Anexo C:

1.4 Incorporación de Baselines Modernos

PISA (RecSys 2024): Por sugerencia del corrector, se incorporará este baseline avanzado (Tran et al., 2024). PISA combina mecanismos de Transformers con los postulados de la teoría cognitiva ACT-R (Adaptive Control of Thought—Rational), modelando el decaimiento de la activación de un ítem en la memoria del usuario a lo largo del tiempo y su refuerzo ante nuevas exposiciones. Esto permite capturar simultáneamente dinámicas secuenciales y huellas temporales de repetición. Aún no se ha evaluado su desempeño, pero está en consideración para los próximos pasos.

RepeatNet: Se mantiene como punto de comparación clásico de la literatura (Ren et al., 2019). No obstante, para optimizar esfuerzos de ingeniería, se utilizará la implementación estandarizada de la biblioteca RecBole.

2. Experimentación Realizada y Evaluación Intermedia

2.1 Configuración Experimental y Datos

La evaluación intermedia se ejecutó sobre el subconjunto H1 de Last.fm 1K Users, que incluye 200,000 interacciones, 100 usuarios y 88,234 ítems únicos. El pipeline experimental automatizado se compone de tres módulos bajo el directorio src/: `baselines.py` (implementaciones con interfaz común de ajuste y recomendación), `evaluation.py` (partición temporal y métricas) y `run_baselines.py` (script ejecutable que genera los resultados reproducibles).

El diseño adopta un protocolo temporal leave-one-out, reservando la última interacción cronológica de cada usuario como dato de prueba y destinando los eventos previos al entrenamiento. Esto genera un conjunto de entrenamiento de 199,900 interacciones y un conjunto de prueba de exactamente 100 eventos. El recomendador propone una lista de 10 ítems candidatos sobre el catálogo completo, sin filtrar el historial del usuario. Mantener el historial es indispensable en el paradigma repeat-aware, ya que remover los ítems consumidos penalizaría por construcción cualquier intento de predecir la repetición. A los tres modelos de referencia iniciales se añadió una cuarta variante, SimpleRepeatRecency, que ordena el historial por criterio cronológico inverso. Esta variante permite contrastar experimentalmente la validez de la hipótesis de localidad temporal.

3. Análisis Preliminar de los Resultados Obtenidos

3.1 Análisis Descriptivo Avanzado de Patrones Temporales

El subconjunto H1 registró una **tasa global de repetición del 60.43%**, la cual desciende al 33% al aislar la última interacción bajo el protocolo leave-one-out. Esta brecha demuestra que los eventos terminales del historial poseen un carácter más exploratorio que el promedio.

Por otro lado, los intervalos entre repeticiones exhiben una marcada asimetría: **el 50% ocurre dentro de las primeras 27.04 horas** y el 25% en menos de 1.74 horas. Esta concentración evidencia un fenómeno de **localidad temporal** (consumo en ráfagas) que coexiste con una cola larga de ítems rezagados. Dada la alta heterogeneidad inter-usuario detectada (tasas individuales entre 3.2% y 98.2%), se justifica que la ventana óptima de repetición en T-BPR sea dinámica y calibrada por percentiles individuales, sustentando el enrutamiento aprendido del Modelo Híbrido. Finalmente, la correlación de Spearman entre frecuencia histórica y reaparición resultó débil (0.284), confirmando que la frecuencia absoluta pierde la señal temporal necesaria para la predicción.

Estadístico	Valor (h)
Mediana (p_{50})	27.04
p_{25}	1.74
p_{75}	160.67
Media	175.05
Máximo	13 503.65

Tabla 1: Intervalos entre repeticiones sucesivas (en horas)

3.2 Rendimiento de Baselines bajo Protocolo LOO Temporal

Los resultados cuantitativos obtenidos a través de la ejecución del script `src/run_baselines.py` se detallan de forma consolidada en la Tabla 2:

Modelo	Recall@10	nDCG@10	MRR	Repeat Ratio	Hits
Random	0.0000	0.0000	0.0000	0.009	0/100
MostPopular	0.0000	0.0000	0.0000	0.055	0/100
SimpleRepeat-Freq	0.0000	0.0000	0.0000	0.055	0/100
SimpleRepeat-Recency	0.0900	0.0667	0.0597	1.000	9/100

Tabla 2: Matriz comparativa de rendimiento intermedio de modelos de referencia

3.3 Discusión y Lectura Crítica de Hallazgos

Insuficiencia de la Frecuencia Histórica (Hallazgo 1): El modelo SimpleRepeat-Freq obtuvo un rendimiento nulo (0 aciertos). El análisis del desglose reveló que el ítem de prueba ocupa la posición mediana 221 en el ordenamiento por frecuencia del historial del usuario, y solo en un caso perteneció al top 10. Esto demuestra que la métrica intuitiva del "ítem más escuchado" carece de poder predictivo inmediato para el siguiente consumo secuencial.

Validación de la Recencia y Localidad Temporal (Hallazgo 2): Al estructurar las recomendaciones bajo el criterio de recencia cronológica inversa, SimpleRepeat-Recency alcanzó un Recall@10 de 0.0900. De los 9 aciertos totales, 5 se ubicaron exactamente en la primera posición del ranking. Este resultado constituye una confirmación experimental directa de la hipótesis de localidad temporal.

Diagnóstico mediante Repeat Ratio (Hallazgo 3): Los valores del Repeat Ratio delimitan el espectro de comportamiento de los modelos. Mientras Random registra un 0.9% y los modelos basados en popularidad o frecuencia un 5.5%, SimpleRepeat-Recency se sitúa en un 100% al recomendar únicamente elementos del historial. El objetivo de los modelos avanzados consiste en calibrar dinámicamente este indicador hacia el 33% empírico observado en la última interacción.

Incompetencia de Enfoques no Personalizados (Hallazgo 4): Las estrategias basadas en aleatoriedad y tendencias globales (MostPopular) fueron incapaces de generar aciertos. El sesgo de popularidad hacia artistas comerciales se disuelve al evaluar usuarios intensivos con catálogos amplios y personalizados, lo que se constata con una mediana de 1,105 ítems únicos por usuario (ver Anexo B).

Esto fundamenta la decisión de incorporar un filtrado colaborativo personalizado (iALS) en el módulo de exploración del modelo híbrido.

4. Problemas Identificados durante el Proceso

Inconsistencias metodológicas en H1: La evaluación inicial usaba una partición aleatoria y filtraba el historial del catálogo de candidatos. Ambos criterios eran incompatibles con el paradigma repeat-aware, ya que destruían la secuencia temporal e impedían recomendar repeticiones. Se resolvió migrando a un módulo de evaluación temporal independiente (src/evaluation.py), lo que explica la falta de comparación directa con H1.

Saturación de memoria en preprocesamiento: El archivo original de 2.5 GB colapsaba la RAM en equipos con menos de 16 GB debido a una carga ineficiente de tipos de datos en pandas. El problema se solucionó con la lectura por fragmentos y la migración incremental a Parquet detallada en la sección 1.1.

Desviación del cronograma de H1: Los compromisos de entrenar RepeatNet y evaluar su sensibilidad no se cumplieron en esta etapa. El esfuerzo se redirigió a prioridades críticas: asegurar un pipeline de evaluación correcto, optimizar el uso de memoria RAM y formalizar matemáticamente los modelos propios (T-BPR e Híbrido). Estas tareas pendientes fueron reprogramadas para H3.

5. Revisión del Plan Propuesto y Justificación de Ajustes

5.1 Pivote Metodológico de las Contribuciones

Pivote metodológico: Tras profundizar en la literatura, se elevó a T-BPR y al Modelo Híbrido al estatus de contribuciones principales propias del grupo. Modelos como RepeatNet y PISA pasan a ser baselines avanzados de validación utilizando sus implementaciones oficiales, lo que reduce riesgos de ingeniería y permite concentrar esfuerzos en el diseño propio. Esto se alinea con el hallazgo de que la frecuencia absoluta por sí sola es una señal débil para la predicción.

Cronograma actualizado hacia H3: El plan de trabajo se estructuró en cuatro semanas diferenciadas:

- Semana 1: Implementación final de T-BPR y análisis de sensibilidad de hiperparámetros.
- Semana 2: Entrenamiento del clasificador del Modelo Híbrido y estudio de ablación de características.
- Semana 3: Ejecución final de los baselines avanzados (RepeatNet y PISA) sobre ambos datasets.
- Semana 4: Redacción del artículo definitivo en formato ACM (máximo 8 páginas) y preparación del póster de defensa.

6. Bibliografía

Anderson, J. R., & Lebiere, C. (1998). *The atomic components of thought*. Lawrence Erlbaum Associates.

Benson, A. R., Kumar, R., & Tomkins, A. (2016). Modeling user consumption sequences. En *Proceedings of the 25th International Conference on World Wide Web (WWW '16)*, pp. 11-19). <https://www.cs.cornell.edu/~arb/papers/sequences-www2016.pdf>

- Ren, P., Chen, Z., Li, J., Ren, Z., Ma, J., & de Rijke, M. (2019). RepeatNet: A repeat aware neural recommendation machine for session-based recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 4806–4813. <https://doi.org/10.1609/aaai.v33i01.33014806>
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. En *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence* (UAI '09, pp. 452–461).
- Tran, V.-A., Salha-Galvan, G., Sguerra, B., & Hennequin, R. (2024). Transformers meet ACT-R: Repeat-aware and sequential listening session recommendation. En *Proceedings of the 18th ACM Conference on Recommender Systems* (RecSys '24).
- Zhao, W. X., Mu, S., Hou, Y., Xue, J., Pan, J., Zhou, K., Chen, Y., Wang, X., Ji, R., Li, R., Zhang, J., Bao, S., & Wen, J.-R. (2021). RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. En *Proceedings of the 30th ACM International Conference on Information and Knowledge Management* (CIKM '21).

7. Anexos

Anexo A: Análisis y Mitigación de Riesgos Técnicos

Incertidumbre e Integración de Modelos de la Literatura: Los modelos *PISA* y *RepeatNet* representan los componentes con mayor riesgo debido a posibles incompatibilidades de software. Aunque el uso de las librerías oficiales de Deezer y RecBole reduce este riesgo, ambos códigos requieren ser adaptados para interactuar con nuestro módulo de evaluación (src/evaluation.py).

Plan de Mitigación: Se ha agendado una fase de integración progresiva en la semana 4 del cronograma. En caso de presentarse un bloqueo técnico con las librerías de referencia, se ha definido una contingencia (*fallback*) que consiste en la codificación propia de una versión simplificada de PISA (un Transformer secuencial acoplado a una función matemática de decaimiento temporal basada en ACT-R).

Esfuerzo de Replicación en el Nuevo Dominio de Datos: La incorporación de *Amazon Grocery* exige ejecutar de manera íntegra todas las fases de limpieza y estructuración de datos.

Plan de Mitigación: Con el propósito de acelerar el proceso, se reutilizará la arquitectura de procesamiento por fragmentos (*chunked*) y almacenamiento en Parquet desarrollada para Last.fm, estimando un esfuerzo técnico dedicado de 2 a 3 días por parte de un integrante del equipo.

Error de Muestreo por Reducción del Tamaño de Prueba: La evaluación intermedia reportada en la sección 4 hace uso exclusivo de 100 usuarios (100 puntos de prueba bajo LOO), lo cual limita la potencia estadística de los resultados.

Plan de Mitigación: Para la entrega final H3, el pipeline experimental se escalará con el fin de evaluar a los 992 usuarios que integran el dataset completo de Last.fm, incorporando la automatización de intervalos de confianza al 95% mediante remuestreo por *bootstrap* a nivel de usuario.

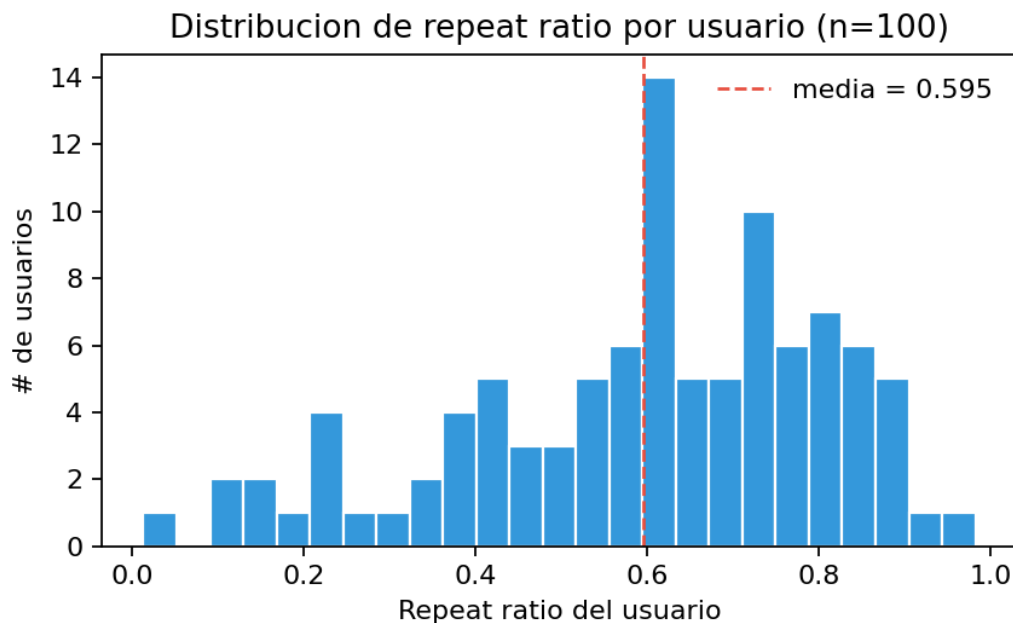
Restricciones de Hardware y Capacidad de Cómputo: El entrenamiento de los modelos avanzados basados en redes neuronales secuenciales (*PISA* y *RepeatNet*) sobre el volumen total de datos puede demandar recursos de aceleración gráfica (GPU) que superen las capacidades locales del equipo.

Plan de Mitigación: En caso de experimentar demoras excesivas, se gestionará el uso de entornos en la nube como *Google Colab* o, de ser necesario, se solicitará acceso formal al clúster de servidores de procesamiento de la universidad.

Anexo B: Resumen de Figuras del Análisis Descriptivo (EDA)

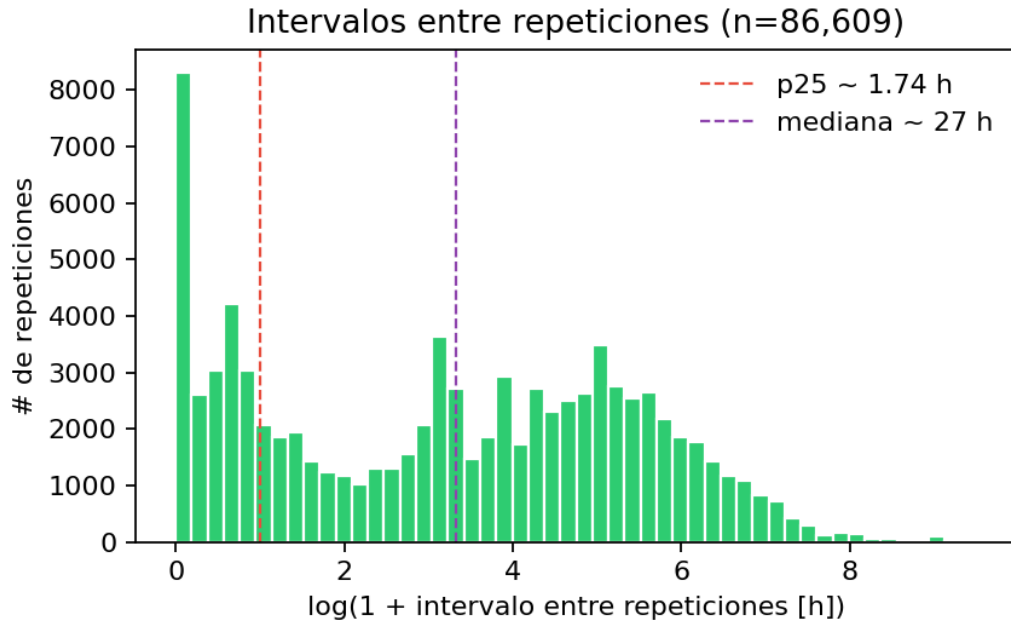
(Nota: Estos gráficos se generan de manera automática ejecutando el script `notebooks/eda_figures.py` sobre el subset de datos H1, encontrándose las imágenes almacenadas en el directorio `docs/figures/` del repositorio).

1. Figura 1 (Distribución del Repeat Ratio individual por usuario, n=100):.



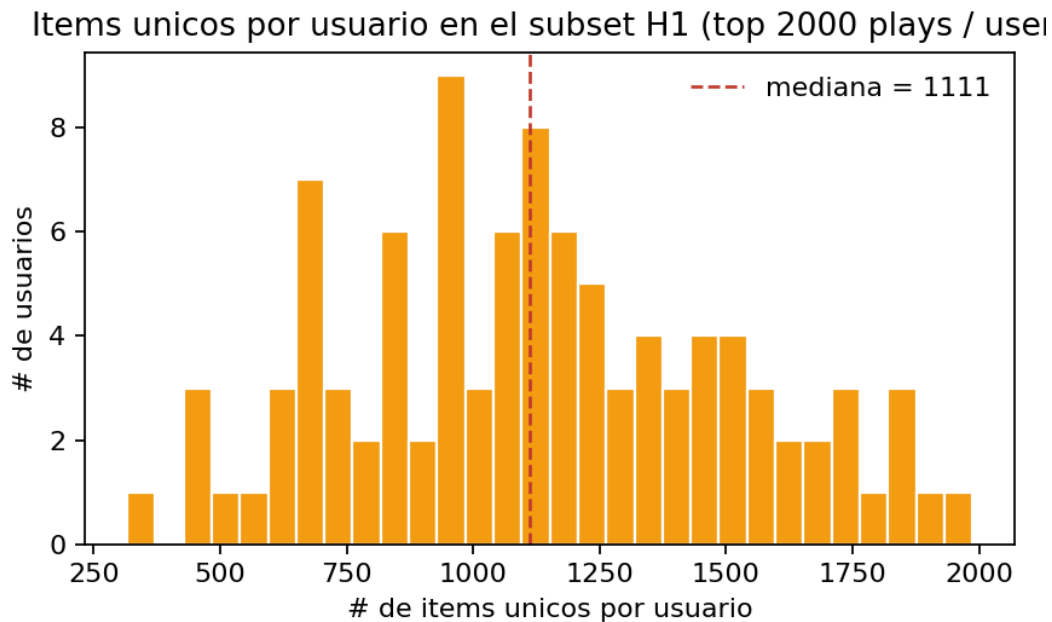
Repeat ratio por usuario (n = 100). La media (línea roja) coincide con la tasa global del 60.43 % discutida en §3.1; la dispersión visualiza la heterogeneidad de §3.3.

2. Figura 2 (Distribución de frecuencias del logaritmo del intervalo temporal entre repeticiones):



Distribución de $\log(1 + \text{intervalo})$ entre repeticiones ($n = 87\,985$). Las líneas verticales marcan $p_{25} \approx 1.74$ h y la mediana ≈ 27 h. La masa concentrada a la izquierda materializa la “localidad temporal” / ráfagas de §3.2.

3. **Figura 3 (Distribución del volumen de ítems únicos identificados por usuario dentro del subset H1):**



Número de ítems únicos por usuario en el subset H1. La mediana $\approx 1\,105$ sobre 2\,000 plays por usuario confirma que el subset captura usuarios con catálogos personalizados amplios, lo que explica el bajo desempeño de MostPopular reportado en §4.3 Hallazgo 4.

Anexo C: Diagrama de Arquitectura del Modelo Híbrido

